

信息学竞赛 AI 自动造数据与数据校验 工具

后端最小可用原型设计报告

版本：V1.0-MVP

设计范围：后端业务闭环与 Dify workflow

设计目标：指导最小可用原型实现与验收

编制日期：2026 年 7 月 13 日

目录

1 设计目标与 MVP 边界

本报告以需求方确认报告和流水线确认文档为依据，设计一个能够完成单道普通信息学竞赛题数据生成的后端最小可用原型。系统接收题目描述、C++ 标程代码和难度等级，经过 AI 分析、用户确认、代码生成、沙箱执行与数据校验，最终导出生成器、校验器和配对的输入输出文件。

MVP 只验证核心业务闭环，不追求完整平台能力。难度等级仅用于项目归档，不传递给 AI，也不影响生成策略。

1.1 MVP 必须实现

- 单题项目创建与本地保存。
- C++ 标程上传或粘贴，以及 Docker 编译检查。
- 阶段 3 的输入结构分析、问题反馈和共同确认。
- 阶段 4 的子任务、数据范围、测试点数量、复杂度和特殊测试点规划。
- 阶段 5 的 `generator.cpp`、`validator.cpp` 生成、修改、种子试运行和共同确认。
- 使用 Python 实现后端，并通过 Python 封装调用 Dify Workflow 接口。
- 提供基于 Compose bootstrap 容器的 Dify 一键初始化；用户无需预先安装、注册或配置本地 Dify。
- 复杂输入结构使用仓库内的 `jngen` 构建，不由 AI 手写底层图、树、排列等构造算法。
- `generator` 和 `validator` 以仓库内 `testlib` 的标准示例为模板生成和编译。
- Agent 工作流及所有 C++ 编译运行均在 Docker 容器内执行，并预先配置 `Dockerfile`。
- Docker 内批量生成输入，使用 `testlib validator` 校验，并由标程生成输出。
- 仅导出生成器、校验器及最终 `.in/.out` 文件。

1.2 MVP 明确不实现

- 不实现前端界面；后端只提供可供后续前端调用的 HTTP API。
- 业务后端不实现数据库、用户账号、多租户、权限、消息队列和分布式任务；Dify 自带的数据库与缓存仅作为其容器化运行依赖，不由本项目开发或访问。
- 不实现 OJ 对接、在线提交、比赛管理、课堂管理、SPJ 和交互题。
- 不生成标准程序，不导出需求报告、质量报告或其他报告文件。

- 不自动替代阶段 3、4、5 的用户确认。

2 总体架构

后端采用 Python 单体服务，项目状态和文件保存在本地工作目录。Dify 不是用户需要预装的软件，而是项目随附的容器化运行依赖：Compose bootstrap 任务从网络拉取固定提交和锁定镜像，启动 Dify 服务并导入项目工作流。Python 服务随后通过 Dify Workflow 接口提交 AI 任务。C++ 编译和运行进入独立 runner 容器。业务数据不进入 Dify 内部存储；Dify 自带的数据库、缓存和队列作为其部署黑盒使用。该结构足以完成 MVP，同时避免为本项目另行引入数据库、缓存和独立任务队列。



组件	MVP 职责
Python/FastAPI 应用	提供项目、阶段执行、确认、试运行、批量生成和导出接口；应用本身运行在 backend 容器中。
项目服务	维护单题项目目录、当前阶段、确认状态和结构化 JSON 文件。
Dify Python 客户端	使用 Python HTTP 客户端调用容器内 Dify Workflow，提交任务要求、上下文和当前结果，取得任务结果与自检确认字段。
Agent 工具网关	接收 Agent 的结构化工具请求，按任务类型、阶段和白名单校验后调用内部服务；不向 Agent 暴露 Shell、路径、网络或 Docker API。

Dify 容器	由 Compose bootstrap 任务拉取、配置并启动，使用项目固定的 <code>dify/docker</code> Compose 配置运行 workflow 服务；用户无需预装 Dify，Agent 节点不在宿主机执行。
Docker 沙箱服务	基于预配置的 runner 镜像创建一次性容器，编译和运行 solution、generator、validator。
本地 C++ 库	runner 镜像从仓库复制 <code>testlib/testlib.h</code> 与 <code>jngen/jngen.h</code> ，不在运行时从网络下载。
数据集服务	批量生成、校验、去除失败文件、生成标准输出、执行命名和 zip 打包。
本地工作区	保存源代码、阶段结果、运行日志和最终数据；本项目业务状态不使用数据库。

3 八阶段后端流程

阶段	名称	后端处理与通过条件
1	创建项目与填写输入	保存题目描述、C++ 标程和难度等级，三项必填；难度仅归档。
2	标程编译检查	在 Docker 中编译标程。成功后继续；失败时返回日志并保留项目输入。
3	输入结构分析	AI 输出可编辑的输入结构草案和完整问题清单。AI 自检通过且用户确认后继续，否则反复修改和核验。
4	子任务规划	保存可扩展子任务表、固定数据范围表和特殊测试点子表。AI 自检、规则校验和用户确认全部通过后继续。
5	代码草稿与试运行	AI 生成 generator 和 validator。用户可修改代码、传入不同种子反复试运行；AI 自检且用户确认后继续。
6	沙箱编译与批量生成	编译三份 C++ 代码并批量生成输入。生成规则问题返回阶段 5，规划问题返回阶段 4。

7	数据校验与输出生成	validator 通过的输入才保留，再运行标程生成配对输出。规划问题返回阶段 4，generator 或 validator 问题返回阶段 5。
8	汇总与导出	生成 zip，仅包含 generator、validator 和最终输入输出数据。

阶段 3、4、5 是共同确认阶段。后端必须分别保存 `ai_confirmed` 和 `user_confirmed`，只有两者同时为真才能更新到下一阶段。阶段 1、2、6、7、8 默认自动执行，发生错误时停止并返回错误信息。

4 Dify workflow 设计

4.1 最小节点组合

通过 `cli-anything-dify-workflow` 检查，MVP 所需的 `start`、`llm`、`code`、`if-else` 和 `end` 节点均可用，条件分支骨架能够通过 DSL 校验。实际 workflow 只保留以下节点：

1. **Start**：接收任务类型、任务要求、上下文和待检查结果。
2. **Task LLM**：执行输入结构分析、子任务规划、generator 草稿生成或 validator 草稿生成。
3. **Normalize Code**：只把 LLM 输出整理为固定 JSON，不访问网络、文件系统或 Docker；解析失败时返回错误。
4. **Check LLM**：将任务要求、全部上下文和执行结果重新检查，不能直接沿用任务节点的结论。
5. **If/Else**：仅根据检查节点的确认字段分支。
6. **End**：返回通过结果，或返回问题清单供下一轮修改。

Dify 提供的 `human-input` 节点不在 MVP 中使用。用户确认由后端 API 记录，避免 Dify 与应用各自维护一套确认状态。

4.2 工具调用封装与最小权限

任何 Agent 均不获得通用工具、Shell、任意 Python、宿主机文件系统、外部网络或 Docker Engine 权限。Agent 只能在结构化输出中提出白名单工具请求；后端先保存候选

结果，再由 AgentToolGateway 校验并调用内部 SandboxService。工具结果经过裁剪和结构化后写回下一轮上下文，Agent 不直接连接工具服务。

Dify 的 Normalize Code 节点只运行项目版本控制中的固定归一化程序，不执行 Agent 返回的 Python 或 JavaScript，也不根据 Agent 输出动态导入模块。该节点关闭外部网络，输入和输出都按固定 JSON Schema 校验。

任务类型	可请求的封装工具	权限边界
input_structure	无	只读取后端提供的题目、标程读取摘要和用户修改稿。
subtask_plan	无	只读取已确认输入结构和用户表单。
generator_code	compile_generator、 preview_generator	只能引用当前项目已保存的 generator 修订号；预览只能传整数种子。
validator_code	compile_validator、 validate_preview	只能引用当前项目已保存的 validator 修订号和后端生成的预览编号。

工具参数只能包含 revision_id、preview_id、seed 等受控标识，不接受命令文本、编译参数、文件路径、挂载点、镜像名或 URL。网关从当前项目状态解析真实文件和固定编译命令，并强制检查任务类型、当前阶段、修订归属、调用次数、超时、CPU 和内存上限。工具只读已确认上下文并写入临时预览区，不得修改确认状态或最终数据。

Task LLM 和 Check LLM 使用同一权限映射，但每次调用都必须重新授权；Check LLM 不能继承 Task LLM 的隐式权限。所有请求记录 workflow 运行号、工具名、参数摘要、结果状态和耗时，日志不得包含 API Key、源码全文或宿主机路径。越权请求直接返回拒绝结果，不尝试猜测或降级到其他工具。

4.3 统一输入输出契约

```
Input:
{
  "task_type": "input_structure | subtask_plan | generator_code | validator_code",
  "requirements": "current task requirements",
  "context": { "problem": "...", "solution": "...", "confirmed": {} },
  "candidate_result": {}
}
```

```
Output:
{
  "confirmation": "pass | revise | error",
  "result": {},
  "tool_requests": [
    { "name": "compile_generator", "arguments": { "revision_id": "r3" } }
  ],
  "issues": ["clear issue or required revision"]
}
```

只有 `confirmation=pass` 且不存在待执行工具请求时，才能把 `ai_confirmed` 设为真。工具请求经网关执行后必须把结果交给下一轮 Check LLM 复核。`revise` 表示结果可修改，后端保存问题并等待用户或 AI 继续迭代；`error` 表示 workflow 调用或结构化输出失败，本轮结果不得覆盖上一版已保存草案。

4.4 四个任务类型

任务类型	输入	结构化结果
input_structure	题目描述、标程读取逻辑、用户修改稿	字段名、类型、读取顺序、分组、重复关系、字段关系、问题清单。
subtask_plan	已确认输入结构、用户子任务表单	子任务列表、固定范围表、测试点总数、期望复杂度、特殊测试点及问题清单。
generator_code	已确认输入结构与子任务规划、当前 generator、编译或试运行反馈	generator.cpp 和问题清单。
validator_code	已确认输入结构与子任务规划、当前 validator、编译或试运行反馈	validator.cpp 和问题清单。

4.5 按 Agent 职责注入库说明

后端在调用 Dify 前由 `AgentContextProvider` 组装上下文。公共上下文只包含当前任务所需的题目、已确认结果、候选结果和运行反馈；`testlib`、`jngen` 的说明作为独立片段按任务类型附加，不允许把整个仓库或无关库说明发送给所有 Agent。

任务类型	附加说明上下文	明确不附加
input_structure	无；仅分析题目与标程读取逻辑。	testlib、jngen 说明。
subtask_plan	无；仅使用已确认输入结构和规划规则。	testlib、jngen 说明。
generator_code	testlib generator 的随机种子、参数和标准输出约定；jngen 基础说明，以及与输入结构匹配的 Array、Tree、Graph、String 等章节。	testlib validator 说明；与本题输入结构无关的 jngen 章节。
validator_code	testlib validator 的 registerValidation、inf 读取、格式检查和失败报告说明。	jngen 说明及 testlib generator 说明。

说明片段由后端从锁定版本的本地仓库文档和标准示例中提取，整理为短小、可复用的只读文本。generator 与 validator 的 Check LLM 必须获得和对应 Task LLM 完全相同的库说明，以便复核 API 是否正确；普通分析、规划及其自检节点不附加库说明。若输入结构同时包含多类复杂结构，只追加实际涉及的 jngen 章节。

难度等级不进入任何 Dify 任务上下文。

4.6 Python 调用接口

后端使用 Python 3.12。Dify 调用封装为单一的 DifyWorkflowClient，内部使用异步 HTTP 客户端访问 Dify Workflow Service API，不直接导入或修改 Dify 内部模块。最小接口如下：

```
class DifyWorkflowClient:
    async def run(
        self,
        task_type: str,
        requirements: str,
        context: dict,
        candidate_result: dict | None,
    ) -> dict:
        """Return confirmation, result, tool_requests and issues."""
```

```
class AgentToolGateway:
    async def execute(
        self,
        project_id: str,
        stage: int,
        task_type: str,
        request: dict,
    ) -> dict:
        """Authorize and execute one allowlisted tool request."""
```

客户端只读取 `DIFY_BASE_URL`、`DIFY_API_KEY` 和 `DIFY_WORKFLOW_ID` 三个环境变量。这些连接信息由初始化流程生成或写入部署环境，不要求业务用户理解 Dify 配置。API Key 只通过 Docker secret 或环境变量注入，不写入源码、项目 JSON 或日志。请求超时、非成功状态码和无效 JSON 统一转换为 `WORKFLOW_FAILED`。

Python 服务调用 Dify 前必须移除难度等级；调用完成后只接受统一输出契约中的四个字段。`tool_requests` 必须通过名称枚举和逐工具 JSON Schema 校验，未知字段也视为越权请求。Dify 自检未返回 `pass` 时，后端不得自行推断任务已经通过。

5 核心数据结构

5.1 输入结构

```
{
  "fields": [
    {
      "name": "n",
      "type": "integer",
      "order": 1,
      "group": "header",
      "repeat_by": null,
      "relation": "controls the following sequence length"
    }
  ],
  "issues": [],
  "ai_confirmed": false,
  "user_confirmed": false
}
```

变量名必须与标程一致。阶段 4 的数据范围固定表只能从这里生成；若字段缺失或分组错误，必须返回阶段 3 修改。

5.2 子任务规划

```
{
  "subtasks": [
    {
      "id": 1,
      "ranges": [
        { "field": "n", "constraint": "n is between 1 and 1000" }
      ],
      "test_count": 10,
      "expected_complexity": "O(n log n)",
      "special_cases": [
        { "count": 2, "description": "all sequence values are equal" }
      ]
    }
  ]
}
```

每个子任务至少包含数据范围固定表、正整数测试点数量和期望通过的复杂度。特殊测试点子表可为空；非空时每项只含数量和描述，且所有特殊项数量之和不得超过子任务总测试点数量。一个实际测试点最多计入一个特殊项。

5.3 项目状态

```
{
  "project_id": "local-generated-id",
  "current_stage": 3,
  "stage_status": "draft | checking | waiting_user | passed | failed",
  "ai_confirmed": false,
  "user_confirmed": false,
  "last_error": null
}
```

用户修改阶段 3、4、5 的草案后，后端必须立即将该阶段的两个确认状态重置为假，并重新执行 AI 检查。

6 本地文件工作区

```
storage/{project_id}/
  project.json
  source/
```

```
    solution.cpp
state/
    input_structure.json
    subtask_plan.json
    code_review.json
generated/
    generator.cpp
    validator.cpp
preview/
data/
    1_1.in
    1_1.out
    1_2.in
    1_2.out
    2_1.in
    2_1.out
logs/
export/
    dataset.zip
```

输入输出文件使用“子任务号 _ 内部编号”命名。两个编号均为无前置零的自然数，同一测试点的 `.in` 和 `.out` 必须同名配对。

写入 JSON、代码或数据文件时先写临时文件，再执行原子替换，避免任务中断后留下半写入状态。所有路径都必须由项目 ID 和后端生成的文件名构造，不接受用户提交的任意宿主机路径。

7 HTTP API 设计

接口	MVP 用途
POST /api/projects	创建项目并提交题目描述、标程代码和难度等级。
GET /api/projects/{id}	获取当前阶段、确认状态、错误和可展示结果。
POST /api/projects/{id}/solution/compile	在 Docker 中编译标程并返回日志。
POST /api/projects/{id}/stages/{3 4 5}/run	将当前上下文提交 Dify，保存任务结果和自检确认字段。

PUT /api/projects/{id}/stages/{3 4 5}/draft	保存用户修改稿，并清除该阶段已有确认状态。
POST /api/projects/{id}/stages/{3 4 5}/confirm	记录用户确认；仅在 AI 已确认时允许该阶段通过。
POST /api/projects/{id}/preview	使用指定种子试运行 generator，返回输入预览和 validator 结果。
POST /api/projects/{id}/build	执行阶段 6 和 7：编译、批量生成、校验并生成输出。
GET /api/projects/{id}/export	阶段 8 打包并返回 zip 文件。

所有错误响应统一包含 `code`、`message`、`stage` 和可选 `details`。MVP 采用单进程串行执行，同一项目同一时间只允许一个任务运行，冲突请求返回 409。

8 Docker、testlib 与 jngen 设计

8.1 容器与 Dockerfile

MVP 在开发业务代码前先准备容器配置。仓库不假设用户机器已有 Dify；初始化任务通过 Git 子模块从网络取得固定版本的 Dify 源码。项目以根目录的跨平台 Compose 文件作为唯一启动入口，Dify 原始 Compose 文件仅作为固定版本的配置依据：

```
compose.yaml    （统一入口）
dify/docker/docker-compose.yaml  （Dify 配置基线）
```

本项目额外维护三个 Dockerfile：

文件	固定职责
docker/backend.Dockerfile	通过 ARG BACKEND_BASE_IMAGE 使用锁定的 Python 3.12 补丁版本镜像，安装 FastAPI、ASGI Server、Python HTTP 客户端和 Docker Python SDK，复制后端源码并以非 root 用户启动 API。

<code>docker/runner.Dockerfile</code>	通过 ARG RUNNER_BASE_IMAGE 使用锁定的 Linux C++ 编译环境，安装固定版本的 g++，从仓库复制 testlib/testlib.h 和 jngen/jngen.h 到只读 include 目录，并创建非 root 运行用户。
<code>docker/bootstrap.Dockerfile</code>	提供固定版本的 Git 和初始化程序，在 Linux 容器内完成子模块获取、配置生成、摘要核验、Dify 初始化和工作流导入，不调用宿主机脚本环境。

backend 容器与 Dify 容器位于同一 Docker 网络，通过服务名访问 Dify API。backend 通过 Compose 内的受限 Docker API 代理启动短生命周期 runner 容器，不依赖宿主机特定的 Docker socket 路径。runner 禁用网络，只挂载命名卷中的当前任务目录；任务结束后立即删除。宿主机只负责 Docker 编排和导出目录，不直接执行 Agent 节点或 C++ 程序。

```
docker compose
|-- bootstrap      cross-platform initialization job
|-- backend        Python API and Dify client
|-- docker-api     restricted Docker Engine proxy
|-- dify services  Agent workflow execution
`-- runner jobs    disposable C++ compile/run containers
```

8.2 Dify 初次部署

项目使用 `docker compose run --rm bootstrap` 作为统一初始化命令。该命令在 Linux bootstrap 容器中执行，在 Bash、PowerShell 和常见图形化终端中的写法一致。部署者只需具备 Docker、Docker Compose、可访问镜像仓库的网络，以及模型服务所需的凭据，不需要提前安装 Dify、Python、Git 或 C++ 工具链。初始化任务按以下顺序执行：

1. 检查 Docker、Compose、网络和可用磁盘空间；条件不满足时给出明确错误并停止。
2. 从网络初始化 Dify、testlib、jngen 子模块，并核对实际提交与报告规定的固定提交一致。
3. 从受版本控制的模板生成 Dify 环境文件和内部随机密钥；模型服务凭据只写入 Docker secret 或部署环境。
4. 按 `docker/images.lock` 拉取并核对全部镜像，然后启动 Dify 依赖服务、API 与 Worker。

5. 等待 Dify 健康检查通过，导入仓库内固定版本的 Workflow DSL，建立后端所需的 Workflow 标识和访问凭据。
6. 启动 backend 与 runner，并执行一次不含真实题目数据的连通性自检。任一步失败都停止后续启动并保留可读日志。

初始化完成后，普通用户只通过本项目 API 使用功能，不进入 Dify 控制台。若更换模型服务凭据，只更新部署密钥并重新执行连通性检查，不重新拉取最新版 Dify。

8.3 跨平台 Docker 兼容

MVP 支持 Linux x86_64、Linux arm64、Intel/Apple Silicon macOS 的 Docker Desktop，以及 Windows 10/11 的 Docker Desktop WSL2 Linux 容器模式。所有业务、Dify 和编译任务统一运行 Linux 容器，宿主系统不参与路径解析、C++ 编译或 Agent 执行。

- backend、bootstrap、runner 及选定的 Dify 服务镜像必须同时提供 linux/amd64 与 linux/arm64 清单；构建使用 Docker Buildx 生成双架构镜像。
- Dify 状态、项目工作区和临时编译目录使用 Docker named volume，避免 Windows 盘符、macOS 目录权限和 Linux UID 差异。仅最终导出目录使用相对路径 `./exports` 绑定挂载。
- Compose 文件不得写入 `/Users/...`、`C:\...` 等宿主机绝对路径，也不得依赖 `host.docker.internal`；服务间通信只使用 Compose 服务名。
- 文本源码和生成的数据文件在容器内统一使用 LF；Windows 检出的 CRLF 文件进入编译前先规范化，避免脚本和 C++ 编译行为差异。
- Docker API 代理只暴露创建、查询和删除 runner 所需的最小接口；各平台通过 Compose 配置连接本机 Docker Engine，backend 只连接代理服务。

8.4 镜像与依赖版本锁定

所有 Docker 镜像必须从配置的网络镜像仓库显式拉取，不使用宿主机上名称相同但来源不明的镜像。项目维护 `docker/images.lock`，逐项记录 backend、bootstrap、runner 及 Dify Compose 各服务镜像的仓库地址、精确版本标签、多架构 manifest digest，并分别记录 linux/amd64 与 linux/arm64 子镜像 digest。Dockerfile 的 FROM 和 Compose 的 image 必须解析到锁定摘要，初始化时同时核对当前平台与摘要是否匹配。

禁止使用 `latest`、`stable`、`main`、仅主版本号或仅次版本号等浮动引用。首次部署先按锁定清单执行 `docker compose pull` 或 `docker pull`，拉取完成后核对实际 RepoDigest；任一镜像缺少、摘要不一致或无法从网络拉取时停止构建和启动，不自动改用其他版本。

Dify、testlib 和 jngen 的源码基线分别固定为当前 Git 子模块提交 d17799825534、1e4e8a24c79c 和 8d1e33be29d6。Dify 各服务必须采用同一基线配置中相互兼容的一组镜像，不允许单独升级某个服务。版本升级只能通过人工修改锁定清单和子模块提交完成，随后重新构建并执行完整验收；运行时不得自动跟随上游最新版。

8.5 generator 标准

生成器必须以本地 testlib 的 generator 示例为标准，使用命令行参数确定随机种子并将单个测试点写到标准输出。对于只包含简单标量或短序列的数据，可以直接采用 testlib 的 `registerGen(argc, argv, 1)` 和 `rnd`。

对于数组、排列、树、图及其他不宜手写的结构，必须使用本地 `jngen.h` 提供的 `Array`、`Tree`、`Graph` 等构造 API。jngen 提供与 testlib 风格兼容的 `registerGen(argc, argv)`、随机数和参数解析能力，因此复杂生成器使用 jngen 入口并继续遵守 testlib 的命令行种子、标准输出和可复现约定。不得在同一源文件中同时调用 testlib 与 jngen 的两个 `registerGen`，也不得让 AI 重新实现 jngen 已提供的结构算法。

AI 生成 generator 后，后端执行静态检查：复杂结构代码必须包含 `jngen.h` 并调用对应结构 API；所有 generator 必须有唯一随机初始化入口、明确参数读取和标准输出。检查不通过时留在阶段 5。

8.6 validator 标准

validator 必须包含仓库内的 `testlib.h`，调用 `registerValidation(argc, argv)`，通过 `inf` 按阶段 3 已确认的读取顺序读取输入，并检查范围、结构关系、行尾和 EOF。不得使用正则表达式或普通字符串拆分替代 testlib 的输入读取和失败报告。

generator 和 validator 均从 runner 镜像内的只读本地头文件编译，避免 AI 生成不同版本的库文件。建议固定编译命令如下：

```
g++ -std=c++17 -O2 -pipe -I/opt/testlib -I/opt/jngen \  
    generator.cpp -o generator  
g++ -std=c++17 -O2 -pipe -I/opt/testlib \  
    validator.cpp -o validator  
g++ -std=c++17 -O2 -pipe solution.cpp -o solution
```

8.7 编译与试运行

标程、generator 和 validator 均使用 runner 镜像中的 g++ 编译。容器禁用网络，只挂载当前项目的临时工作目录，并设置固定的 CPU、内存和执行超时。业务代码不得在

宿主机或 backend 容器中直接运行。

阶段 5 的预览接口接收一个整数种子，运行一次 generator，再运行 validator。用户可以多次更换种子；预览文件不进入最终数据目录。代码编译或运行失败时，后端保存日志并返回阶段 5 继续修改和确认。

8.8 批量生成算法

1. 确认阶段 3、4、5 均已通过，且三份 C++ 代码编译成功。
2. 按子任务顺序和测试点数量生成每个输入文件。
3. 特殊测试点按特殊项数量分别生成；剩余数量作为普通测试点。
4. 每个输入立即运行 validator；失败文件不进入最终数据目录。
5. 对通过校验的输入运行标程并写入同名输出文件。
6. 检查每个子任务的最终合法数量是否达到计划值；不足时停止导出并按原因返回阶段 4 或 5。

8.9 最小错误分类

错误码	含义	处理
COMPILE_FAILED	任一 C++ 文件编译失败	标程返回阶段 2；generator 或 validator 返回阶段 5。
WORKFLOW_FAILED	Dify 调用失败或输出无法解析	保留旧草稿并重试当前阶段。
TOOL_DENIED	工具不在白名单、参数越界或调用者无权使用	不执行请求，记录原因并继续当前阶段核验。
TOOL_FAILED	已授权工具超时、资源超限或内部执行失败	清理临时容器，返回规范化错误并继续当前阶段。
CONFIRMATION_REQUIRED	AI 或用户尚未确认	停留在阶段 3、4 或 5。

GENERATION_FAILED	generator 运行失败或合法数量不足	生成规则返回阶段 5；规划问题返回阶段 4。
VALIDATION_FAILED	输入未通过 validator	记录原因并返回阶段 4 或 5。
SOLUTION_FAILED	标程运行异常或超时	停止阶段 7 并返回错误。
EXPORT_NOT_READY	数据未全部配对或阶段未通过	禁止生成 zip。

9 导出设计

最终 zip 只包含以下内容：

```
generator.cpp
validator.cpp
data/
  1_1.in
  1_1.out
  1_2.in
  1_2.out
  ...
```

不导出 solution、项目配置、运行日志、确认记录或报告文件。导出前必须检查每个输入都有同名输出、文件名符合规则、所有输入均通过 validator，且各子任务数量与已确认规划一致。

10 实现顺序

1. 确定 amd64/arm64 兼容的镜像集合，写入 `docker/images.lock`，从网络拉取并核对各平台 digest。
2. 实现根目录 `compose.yaml` 和 bootstrap 容器，在未安装 Dify 的干净环境中完成子模块拉取、Dify 启动、健康检查和工作流导入。
3. 配置 backend、runner 和 Docker API 代理，并在 Linux、macOS Docker Desktop、Windows Docker Desktop WSL2 环境中完成基础启动测试。
4. 建立 Python/FastAPI 项目骨架、本地项目目录、JSON 状态和阶段门禁。

- 5. 把固定提交的 testlib 与 jngen 固化进 runner 镜像，验证三类 C++ 代码能够编译运行。
- 6. 实现 AgentContextProvider，形成 testlib generator、testlib validator 和按结构分类的 jngen 说明片段及任务映射测试。
- 7. 实现 AgentToolGateway、逐工具参数 Schema、任务权限映射、调用配额和审计日志，并完成越权测试。
- 8. 使用 Python 接入 Dify Workflow, 先打通阶段 3、4, 再分别接入 generator 与 validator 代码任务。
- 9. 完成 generator、validator 代码检查、保存和种子预览。
- 10. 完成批量生成、validator 校验、标程输出、命名配对和 zip 导出。

每一步只在上一项通过自动化检查后继续，不提前实现前端、数据库或并发任务系统。

11 MVP 验收标准

序号	可验证标准
1	能创建项目并保存三个必填输入，且难度等级不出现在 Dify 请求中。
2	在仅预装 Docker（含 Compose）的干净环境中，bootstrap 容器能够从网络取得固定版本的 Dify 并完成启动、健康检查和工作流导入。
3	同一根目录 Compose 配置能在 Linux x86_64/arm64、Intel/Apple Silicon macOS 和 Windows WSL2 Linux 容器模式启动；不要求宿主机安装 Bash、Python、Git 或 g++。
4	Python 后端能够通过 Dify Workflow 接口调用容器内 Agent，并正确处理超时和无效返回。
5	backend 与 runner Dockerfile 可以成功构建；Agent 工作流和所有 C++ 编译运行均不在宿主机执行。
6	所有镜像均能按当前架构的锁定摘要从网络拉取并通过核验；配置中不存在 latest 或其他浮动版本引用。
7	Dify、testlib 与 jngen 使用报告指定的固定子模块提交，升级不会在运行时自动发生。
8	能在 runner 容器中编译 C++ 标程，失败时返回可读日志。

- 9 阶段 3 能输出可编辑输入结构和问题清单，AI 与用户未共同确认时不能继续。
 - 10 阶段 4 能保存至少一个子任务，并校验固定范围表、正整数总数、复杂度和特殊项数量总和。
 - 11 阶段 5 能生成和修改 generator、validator，并能使用不同种子多次预览。
 - 12 复杂数组、排列、树或图的 generator 使用本地 jngen API，不包含手写的底层结构生成算法。
 - 13 validator 使用本地 testlib 标准入口和读取接口；generator 遵守 testlib 的种子、参数和标准输出约定。
 - 14 输入分析和子任务规划 Agent 不接收 testlib、jngen 说明；代码生成与复核 Agent 只接收其职责和输入结构所需的说明片段。
 - 15 AI 每次完成任务后都会使用要求、上下文和执行结果再次检查，并返回确认字段。
 - 16 Agent 不能直接使用 Shell、任意 Python、文件系统、网络或 Docker API；所有工具请求均经过白名单网关、参数 Schema、阶段权限和资源限制校验。
 - 17 对未授权工具、路径、命令、URL、镜像名和额外参数的测试请求均被拒绝，且不会创建容器、修改项目状态或泄露敏感信息。
 - 18 阶段 6 能按已确认数量批量生成输入，所有不可信代码均在 runner 容器中运行。
 - 19 阶段 7 只保留 validator 通过的输入，并由标程生成一一配对的输出。
 - 20 文件按 子任务号 _ 内部编号.in/out 规则命名，编号无前置零。
 - 21 阶段 8 导出的 zip 只包含 generator、validator 和最终输入输出数据。
 - 22 Dify、工具调用、编译、生成、校验或标程运行失败时，后端停止流程并返回明确阶段和原因。
-

12 结论

本方案以 Python 单体后端和容器化执行完成从题目输入到数据包导出的闭环。Dify 容器承担结构化分析、规划、代码草稿和独立自检；Python 服务负责接口、状态门禁与用户确认；runner 容器使用本地 testlib 和 jngen 完成确定性编译、生成与校验。该边界能够支持后续接入前端，同时避免 MVP 阶段引入与核心验证无关的复杂平台能力。